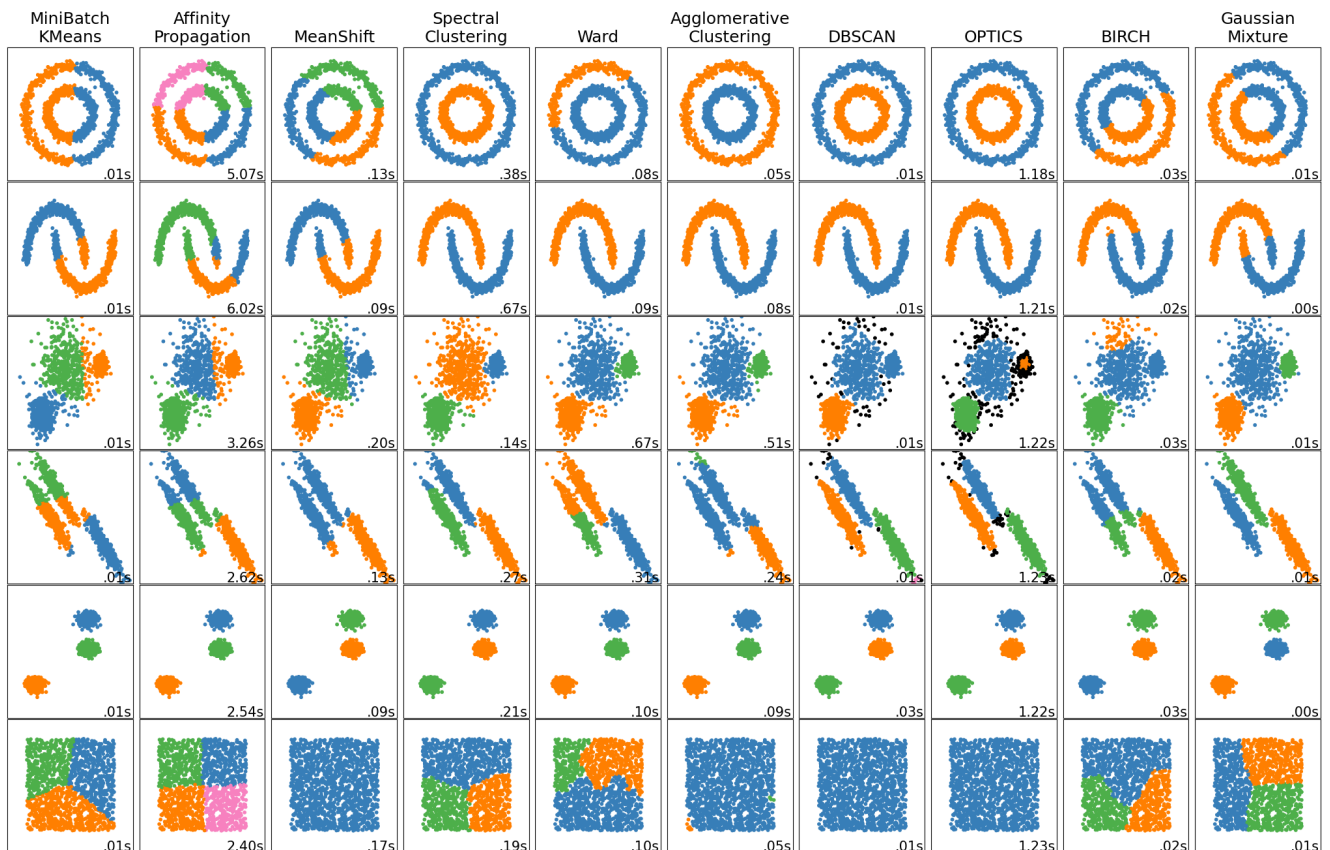


▼ 군집화(Clustering)

- 대표적인 비지도학습 알고리즘
- 레이블이 없는 데이터를 그룹화 하는 알고리즘



```
import numpy as np
import matplotlib.pyplot as plt
plt.style.use(['seaborn-whitegrid'])

from sklearn import cluster
from sklearn import mixture
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
```

▼ 데이터 생성

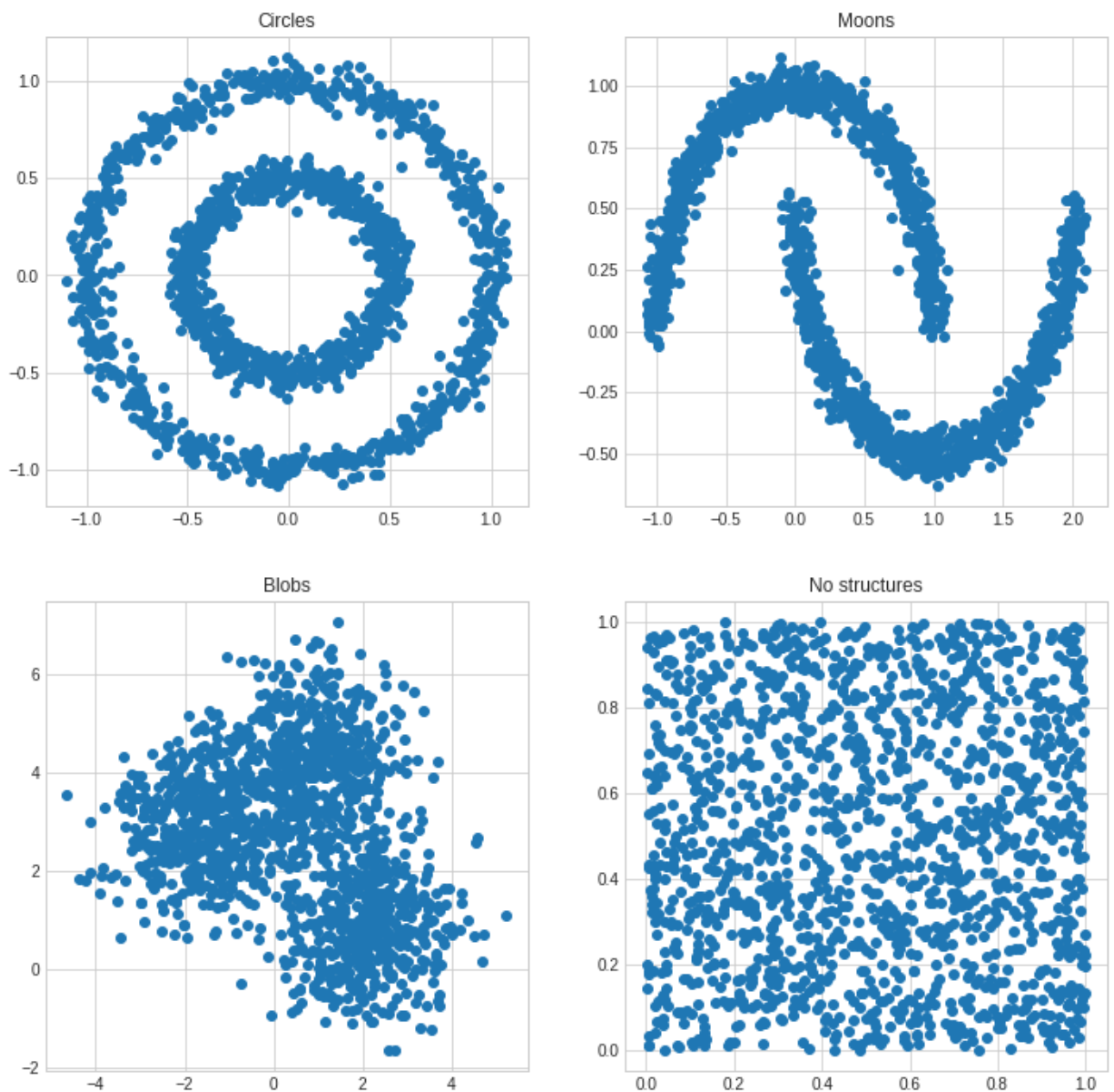
```
def plot_data(dataset, position, title):
    X, y = dataset
    plt.subplot(position)
    plt.title(title)
    plt.scatter(X[:, 0], X[:, 1])
```

```
np.random.seed(0)
n_samples = 1500
random_state = 0
```

```
noise = 0.05
```

```
circles = datasets.make_circles(n_samples=n_samples, factor=0.5, noise=noise, random_state=random_s  
moons = datasets.make_moons(n_samples=n_samples, noise=noise, random_state=random_state)  
blobs = datasets.make_blobs(n_samples=n_samples, random_state=random_state)  
no_structures = np.random.rand(n_samples, 2), None
```

```
plt.figure(figsize=(12, 12))  
plot_data(circles, 221, 'Circles')  
plot_data(moons, 222, 'Moons')  
plot_data(blobs, 223, 'Blobs')  
plot_data(no_structures, 224, 'No structures')
```



```
def fit_predict_plot(model, dataset, position, title):  
    X, y = dataset  
    model.fit(X)
```

```
model.fit(X)
if hasattr(model, 'labels_'):
    labels = model.labels_.astype(np.int)
else:
    labels = model.predict(X)

colors = np.array(['#30A9DE', '#E53A40', '#090707', '#A593E0', '#F6B352', '#519D9E', '#D81159',
ax = plt.subplot(position)
ax.set_title(title)
ax.scatter(X[:, 0], X[:, 1], color=colors[labels])
```

▼ K-평균 (K-Means)

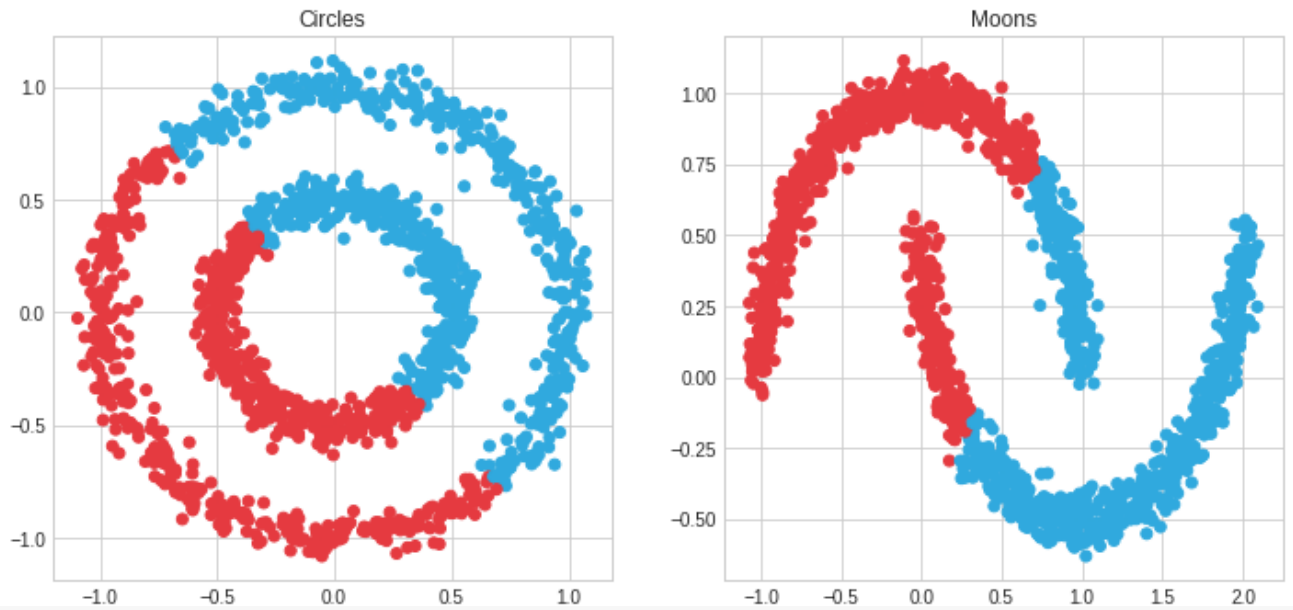
- n 개의 등분산 그룹으로 군집화
- 제곱합 함수를 최소화
- 군집화 개수를 지정해야 한다.
- 각 군집 C 의 평균 μ_j 을 중심점 이라고 함
- 다음을 만족하는 중심점을 찾는것이 목표

$$\sum_{i=0}^n \min_{\mu_j \in C} (||x_i - \mu_j||^2)$$

```
fig = plt.figure(figsize=(12, 12))
fig.suptitle('k-Means')
```

```
fit_predict_plot(cluster.KMeans(n_clusters=2, random_state=random_state), circles, 221, 'Circles')
fit_predict_plot(cluster.KMeans(n_clusters=2, random_state=random_state), moons, 222, 'Moons')
fit_predict_plot(cluster.KMeans(n_clusters=2, random_state=random_state), blobs, 223, 'Blobs')
fit_predict_plot(cluster.KMeans(n_clusters=2, random_state=random_state), no_structures, 224, 'No s
```

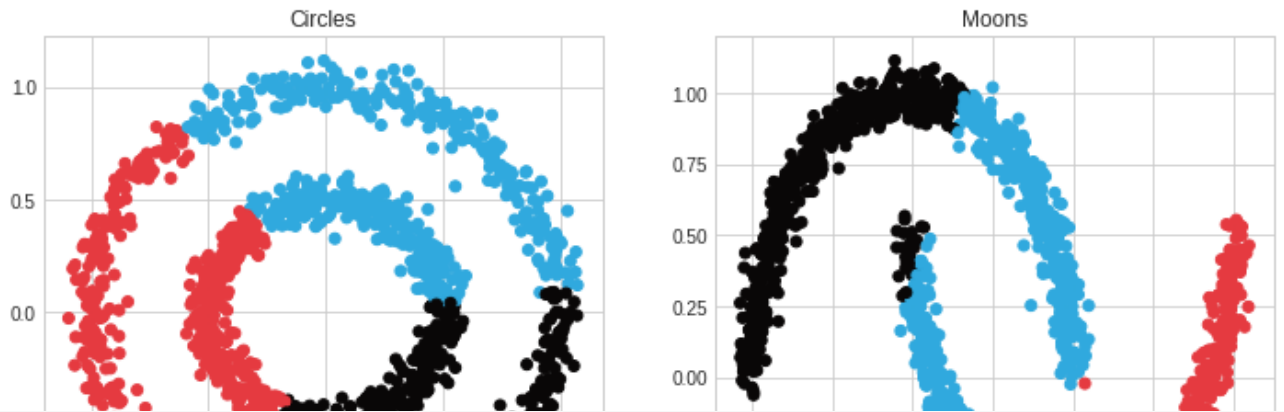
k-Means



```
fig = plt.figure(figsize=(12, 12))  
fig.suptitle('k-Means')
```

```
fit_predict_plot(cluster.KMeans(n_clusters=3, random_state=random_state), circles, 221, 'Circles')  
fit_predict_plot(cluster.KMeans(n_clusters=3, random_state=random_state), moons, 222, 'Moons')  
fit_predict_plot(cluster.KMeans(n_clusters=3, random_state=random_state), blobs, 223, 'Blobs')  
fit_predict_plot(cluster.KMeans(n_clusters=3, random_state=random_state), no_structures, 224, 'No s
```

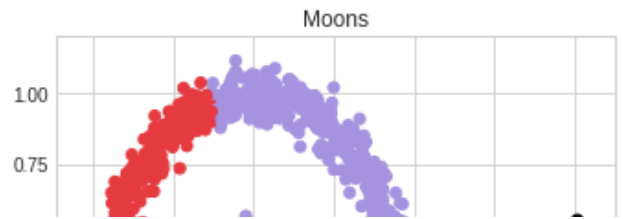
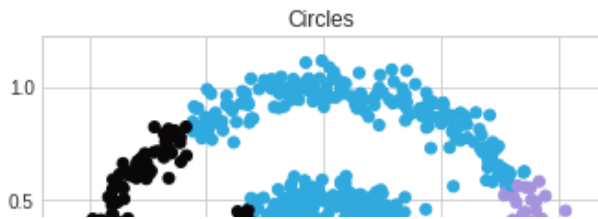
k-Means



```
fig = plt.figure(figsize=(12, 12))  
fig.suptitle('k-Means')
```

```
fit_predict_plot(cluster.KMeans(n_clusters=4, random_state=random_state), circles, 221, 'Circles')  
fit_predict_plot(cluster.KMeans(n_clusters=4, random_state=random_state), moons, 222, 'Moons')  
fit_predict_plot(cluster.KMeans(n_clusters=4, random_state=random_state), blobs, 223, 'Blobs')  
fit_predict_plot(cluster.KMeans(n_clusters=4, random_state=random_state), no_structures, 224, 'No s
```

k-Means



▼ 붓꽃 데이터 군집화

- K-Means 알고리즘을 사용해 붓꽃 데이터를 군집화 해본다.
- 같은 종류의 붓꽃끼리 잘 군집화가 되었다.

```
from sklearn.datasets import load_iris
```

```
iris = load_iris()
```

```
model = cluster.KMeans(n_clusters=3)
model.fit(iris.data)
predict = model.predict(iris.data)
```

```
idx = np.where(predict==0)
iris.target[idx]
```

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0])
```

```
idx = np.where(predict==1)
iris.target[idx]
```

```
array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

```
idx = np.where(predict==2)
iris.target[idx]
```

```
array([1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2])
```

▼ 미니 배치 K-평균 (Mini Batch K-Means)

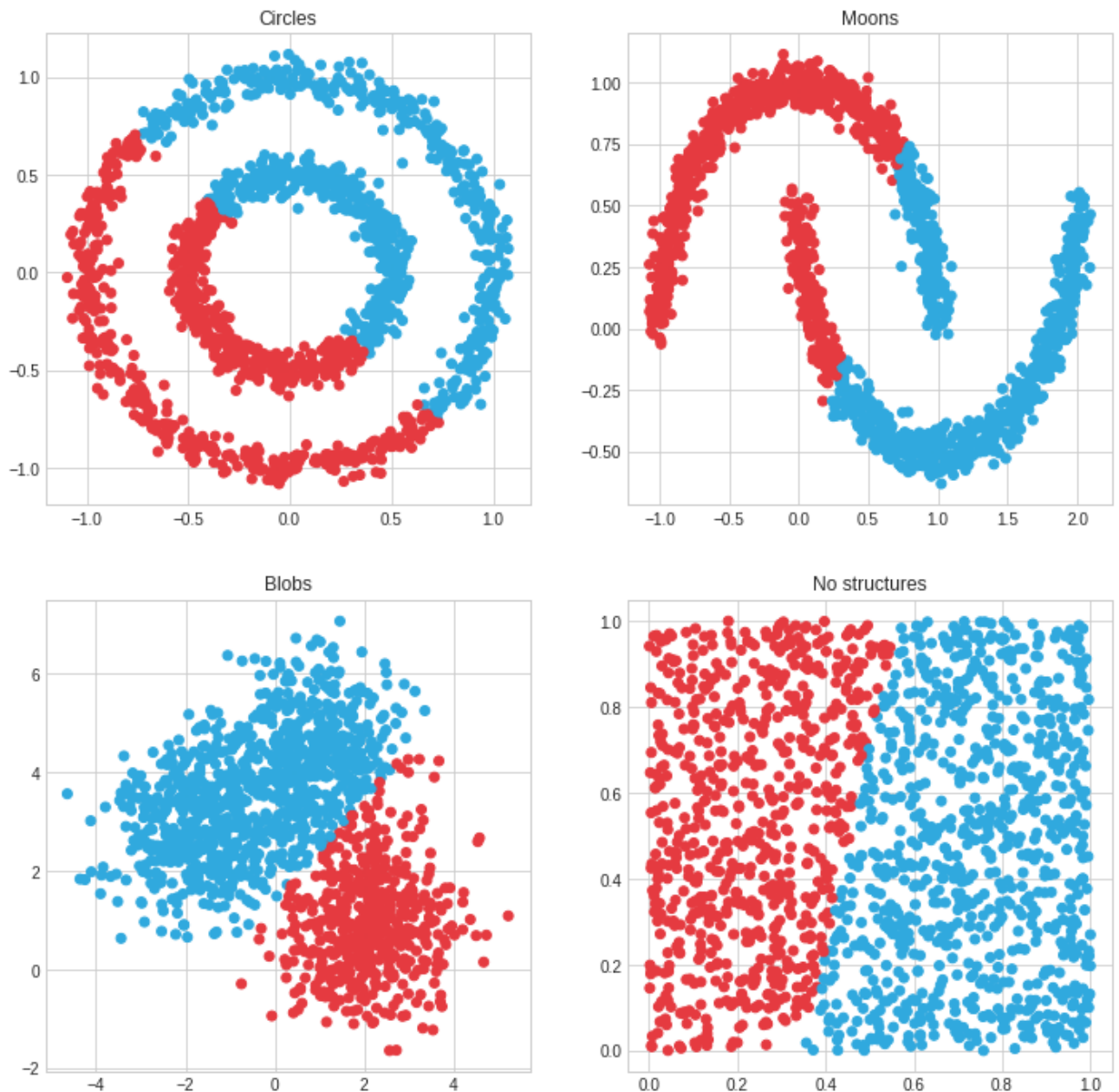
- 배치 처리를 통해 계산 시간을 줄인 K-평균
- K-평균과 다른 결과가 나올 수 있다.

```
fig = plt.figure(figsize=(10, 10))
```

```
fig = plt.figure(figsize=(12, 12))
fig.suptitle('MiniBatch k-Means')
```

```
fit_predict_plot(cluster.MinibatchKMeans(n_clusters=2, random_state=random_state), circles, 221, 'Circles')
fit_predict_plot(cluster.MinibatchKMeans(n_clusters=2, random_state=random_state), moons, 222, 'Moons')
fit_predict_plot(cluster.MinibatchKMeans(n_clusters=2, random_state=random_state), blobs, 223, 'Blobs')
fit_predict_plot(cluster.MinibatchKMeans(n_clusters=2, random_state=random_state), no_structures, 2
```

MiniBatch k-Means

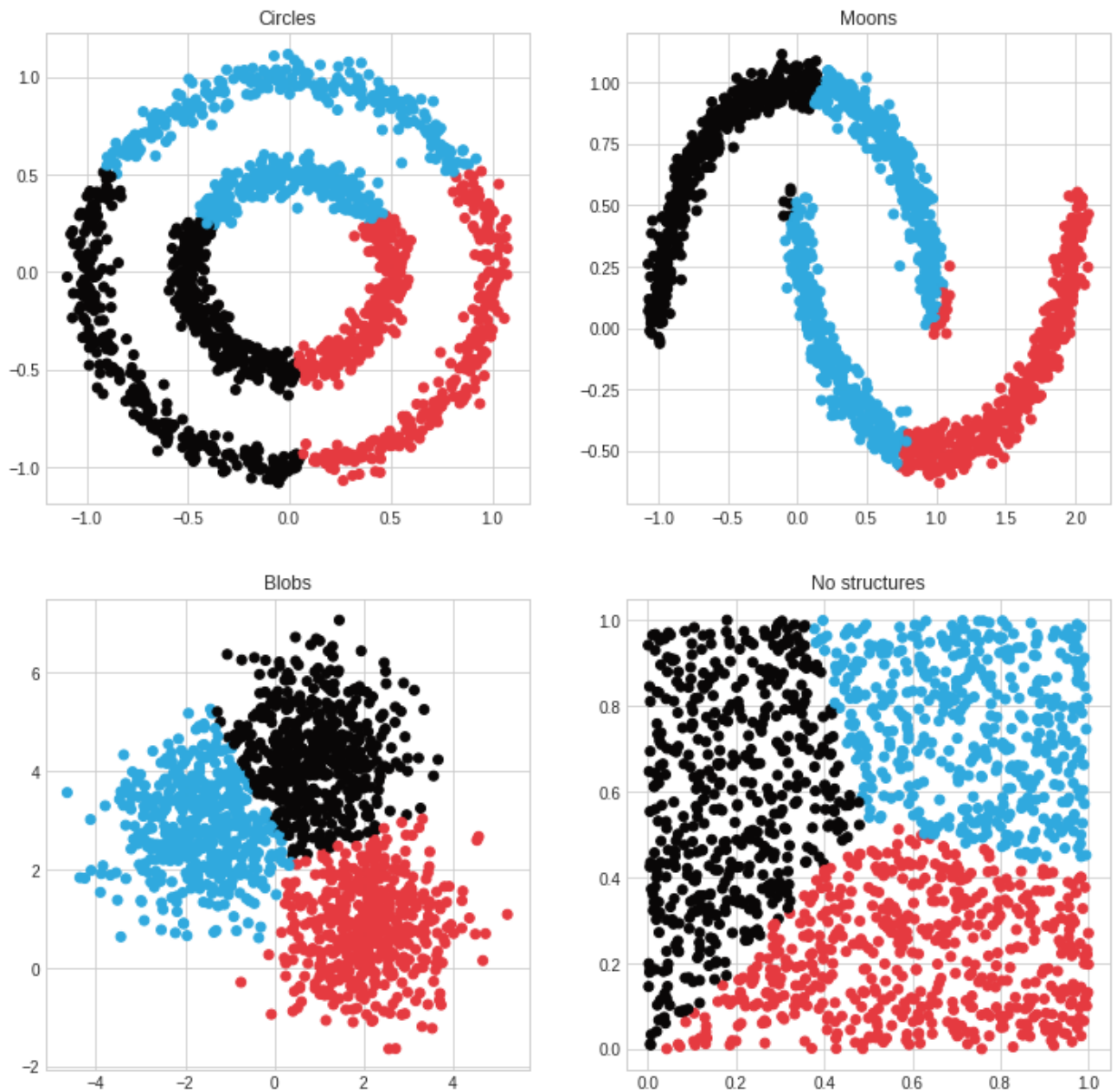


```
fig = plt.figure(figsize=(12, 12))
fig.suptitle('MiniBatch k-Means')
```

```
fit_predict_plot(cluster.MinibatchKMeans(n_clusters=3, random_state=random_state), circles, 221, 'Circles')
fit_predict_plot(cluster.MinibatchKMeans(n_clusters=3, random_state=random_state), moons, 222, 'Moons')
fit_predict_plot(cluster.MinibatchKMeans(n_clusters=3, random_state=random_state), blobs, 223, 'Blobs')
```

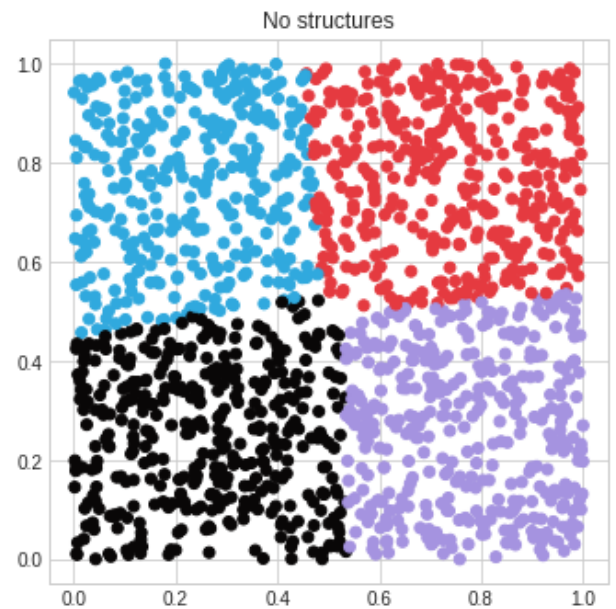
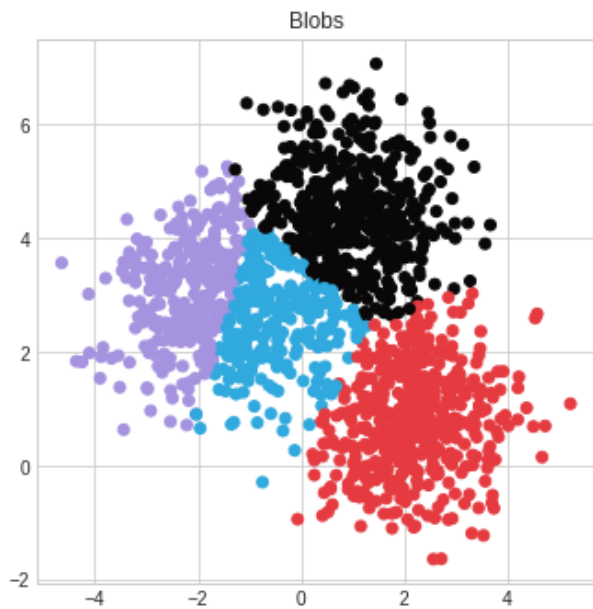
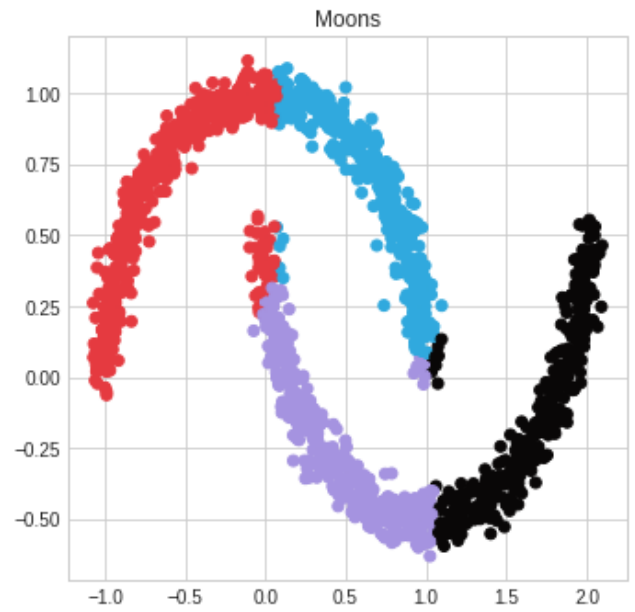
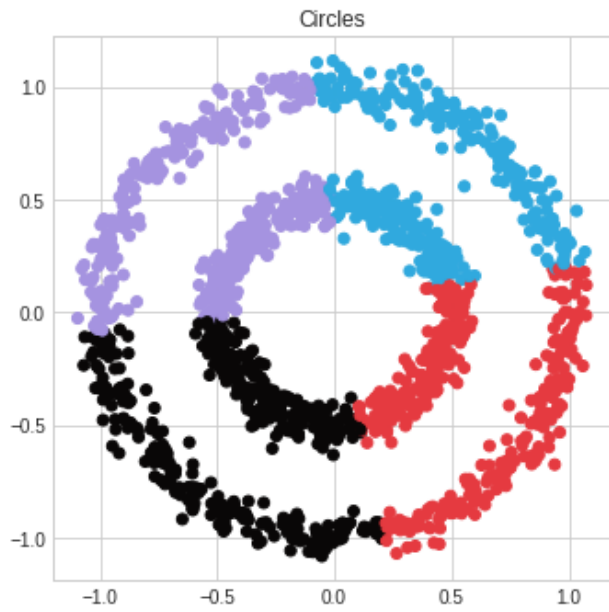
```
fit_predict_plot(cluster.MinibatchKMeans(n_clusters=3, random_state=random_state), blobs, 223, 'Blo')
fit_predict_plot(cluster.MinibatchKMeans(n_clusters=3, random_state=random_state), no_structures, 2
```

MiniBatch k-Means



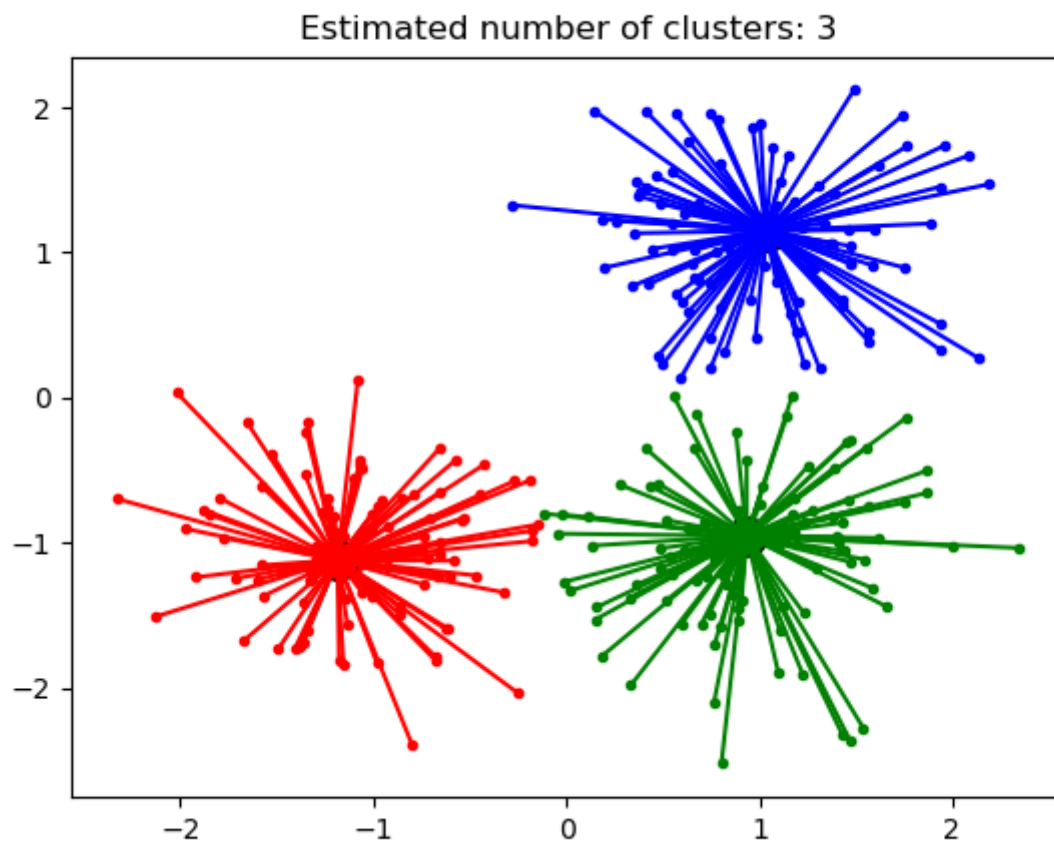
```
fig = plt.figure(figsize=(12, 12))
fig.suptitle('MiniBatch k-Means')
```

```
fit_predict_plot(cluster.MinibatchKMeans(n_clusters=4, random_state=random_state), circles, 221, 'C')
fit_predict_plot(cluster.MinibatchKMeans(n_clusters=4, random_state=random_state), moons, 222, 'Moo')
fit_predict_plot(cluster.MinibatchKMeans(n_clusters=4, random_state=random_state), blobs, 223, 'Blo')
fit_predict_plot(cluster.MinibatchKMeans(n_clusters=4, random_state=random_state), no_structures, 2
```

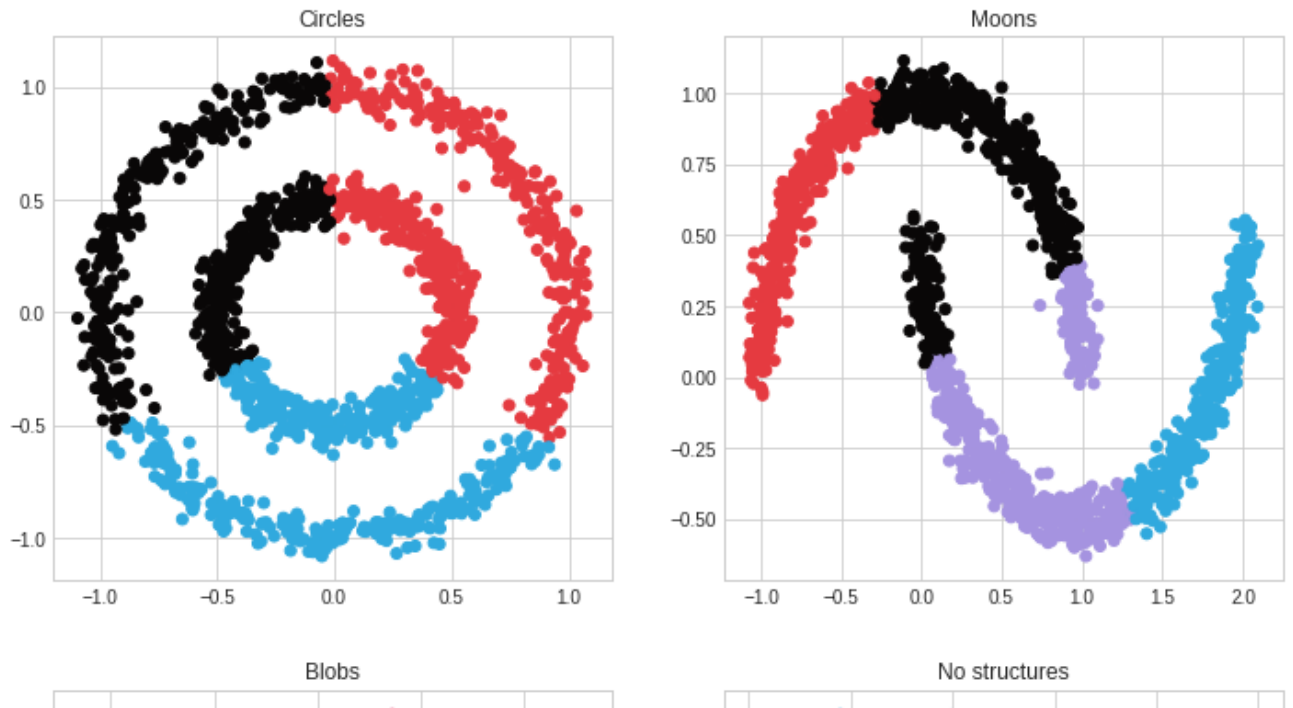
▼ Affinity Propagation

- 샘플 쌍 끼리 메시지를 보내 군집을 생성
- 샘플을 대표하는 적절한 예를 찾을 때까지 반복
- 군집의 갯수를 자동으로 정함



```
fig = plt.figure(figsize=(12, 12))  
fig.suptitle('Affinity Propagation')
```

```
fit_predict_plot(cluster.AffinityPropagation(damping=.9, preference=-200), circles, 221, 'Circles')  
fit_predict_plot(cluster.AffinityPropagation(damping=.9, preference=-200), moons, 222, 'Moons')  
fit_predict_plot(cluster.AffinityPropagation(damping=.9, preference=-200), blobs, 223, 'Blobs')  
fit_predict_plot(cluster.AffinityPropagation(damping=.9, preference=-200), no_structures, 224, 'No
```



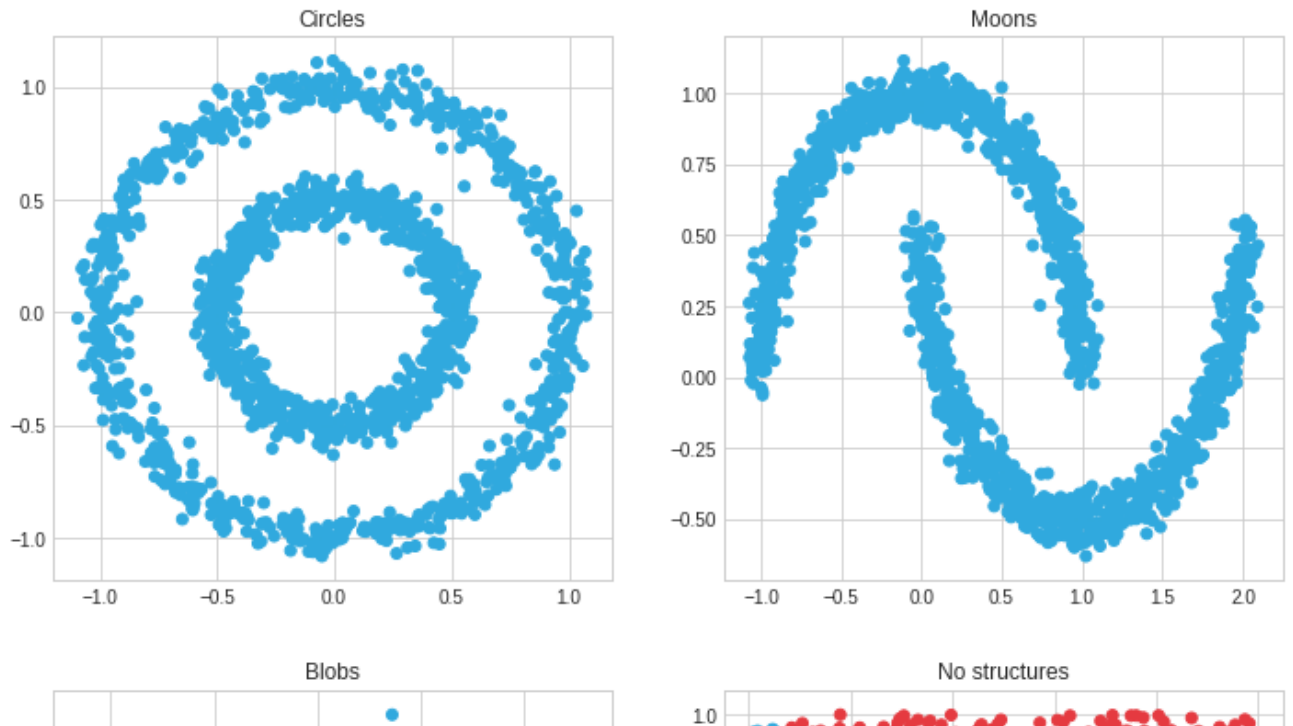
▼ Mean Shift

- 중심점 후보를 정해진 구역 내 평균으로 업데이트

```
fig = plt.figure(figsize=(12, 12))
fig.suptitle('Mean Shift')

fit_predict_plot(cluster.MeanShift(), circles, 221, 'Circles')
fit_predict_plot(cluster.MeanShift(), moons, 222, 'Moons')
fit_predict_plot(cluster.MeanShift(), blobs, 223, 'Blobs')
fit_predict_plot(cluster.MeanShift(), no_structures, 224, 'No structures')
```

Mean Shift

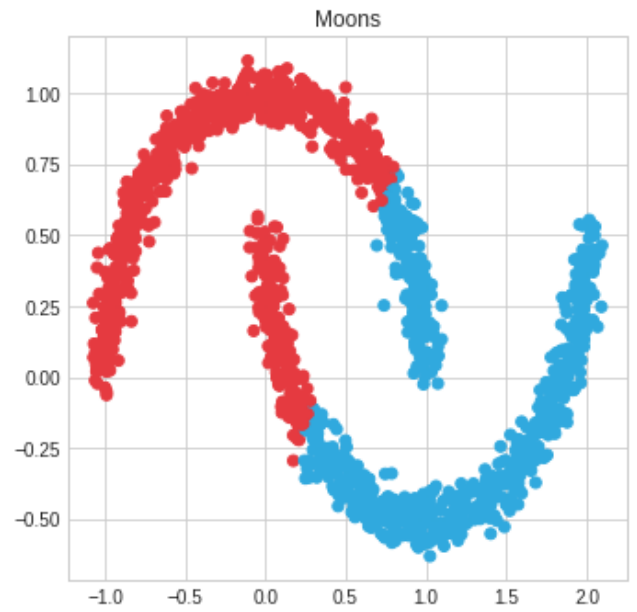
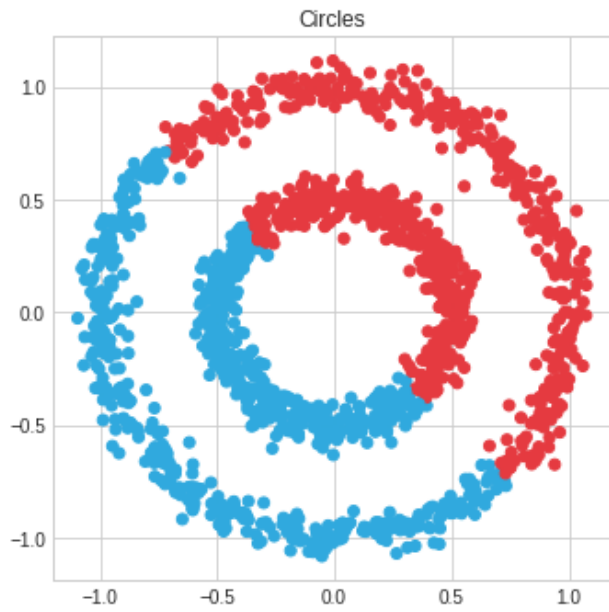


▼ 스펙트럼 군집화 (Spectral Clustering)

```
fig = plt.figure(figsize=(12, 12))
fig.suptitle('Spectral Clustering')
```

```
fit_predict_plot(cluster.SpectralClustering(n_clusters=2, random_state=random_state), circles, 221,
fit_predict_plot(cluster.SpectralClustering(n_clusters=2, random_state=random_state), moons, 222,
fit_predict_plot(cluster.SpectralClustering(n_clusters=2, random_state=random_state), blobs, 223,
fit_predict_plot(cluster.SpectralClustering(n_clusters=2, random_state=random_state), no_structures
```

Spectral Clustering



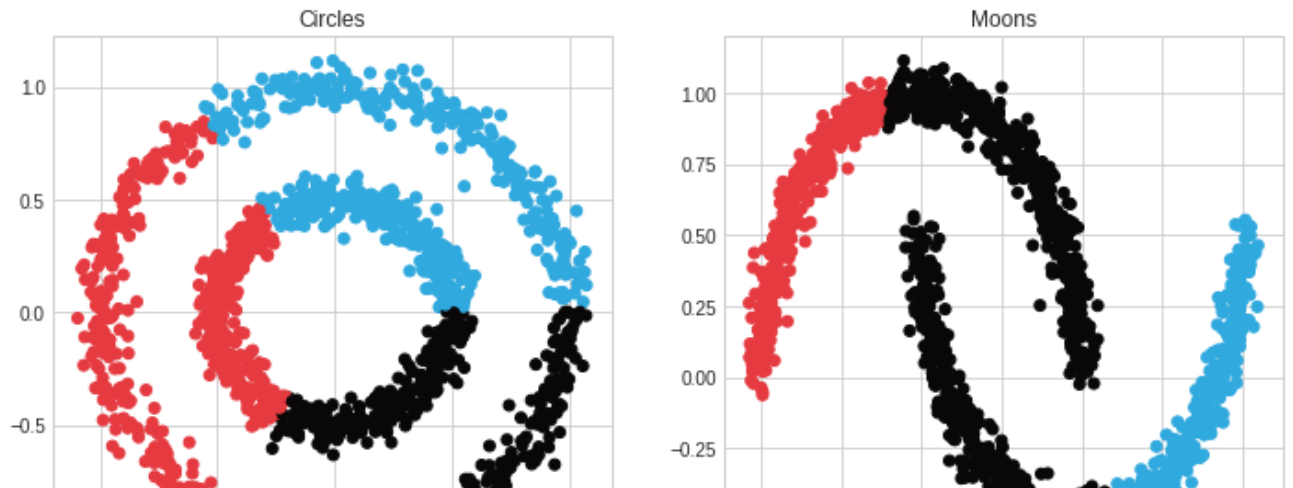
Blobs

No structures

```
fig = plt.figure(figsize=(12, 12))  
fig.suptitle('Spectral Clustering')
```

```
fit_predict_plot(cluster.SpectralClustering(n_clusters=3, random_state=random_state), circles, 221,  
fit_predict_plot(cluster.SpectralClustering(n_clusters=3, random_state=random_state), moons, 222, '  
fit_predict_plot(cluster.SpectralClustering(n_clusters=3, random_state=random_state), blobs, 223, '  
fit_predict_plot(cluster.SpectralClustering(n_clusters=3, random_state=random_state), no_structures
```

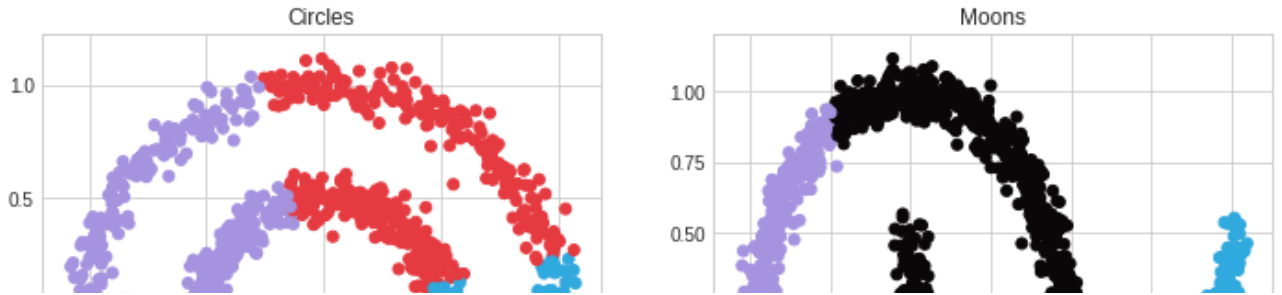
Spectral Clustering



```
fig = plt.figure(figsize=(12, 12))  
fig.suptitle('Spectral Clustering')
```

```
fit_predict_plot(cluster.SpectralClustering(n_clusters=4, random_state=random_state), circles, 221,  
fit_predict_plot(cluster.SpectralClustering(n_clusters=4, random_state=random_state), moons, 222, '  
fit_predict_plot(cluster.SpectralClustering(n_clusters=4, random_state=random_state), blobs, 223, '  
fit_predict_plot(cluster.SpectralClustering(n_clusters=4, random_state=random_state), no_structures
```

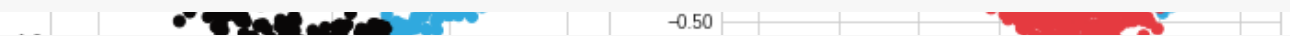
Spectral Clustering



▼ 유방암 데이터 군집화

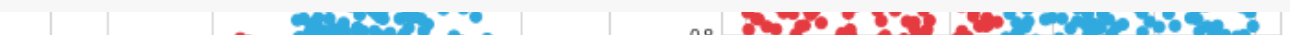


```
from sklearn.datasets import load_breast_cancer
```



```
cancer = load_breast_cancer()
```

```
model = cluster.SpectralClustering(n_clusters=2, eigen_solver='arpack', affinity='nearest_neighbors')
model.fit(cancer.data)
predict = model.labels_
```



```
idx = np.where(predict==0)
cancer.target[idx]
```

```
array([[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1,
        1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 0, 1,
        0, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0,
        1, 1, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1,
        1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0,
        0, 1, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0, 0, 1,
        1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 0, 0, 1,
        1, 0, 0, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 0,
        1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0,
        0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0])
```

```
idx = np.where(predict==1)
cancer.target[idx]
```

```
array([0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
```

[illegible]

▼ 계층 군집화 (Hierarchical Clustering)

```
fig = plt.figure(figsize=(12, 12))
fig.suptitle('Hierarchical Clustering')

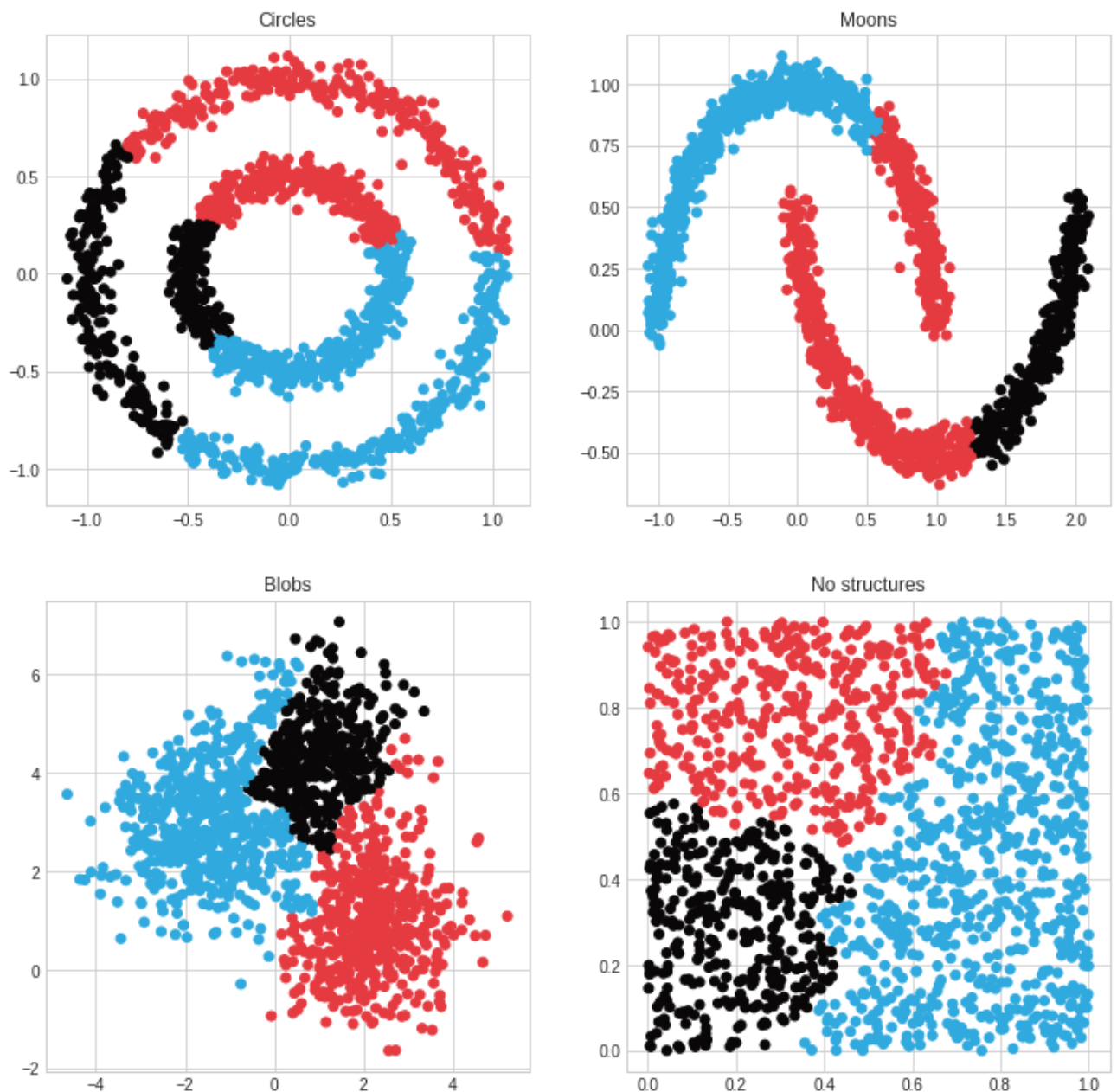
fit_predict_plot(cluster.AgglomerativeClustering(n_clusters=2, linkage='ward'), circles, 221, 'Circles')
fit_predict_plot(cluster.AgglomerativeClustering(n_clusters=2, linkage='ward'), moons, 222, 'Moons')
fit_predict_plot(cluster.AgglomerativeClustering(n_clusters=2, linkage='ward'), blobs, 223, 'Blobs')
fit_predict_plot(cluster.AgglomerativeClustering(n_clusters=2, linkage='ward'), no_structures, 224,
```


Hierarchical Clustering

```
fig = plt.figure(figsize=(12, 12))  
fig.suptitle('Hierarchical Clustering')
```

```
fit_predict_plot(cluster.AgglomerativeClustering(n_clusters=3, linkage='ward'), circles, 221, 'Circles')  
fit_predict_plot(cluster.AgglomerativeClustering(n_clusters=3, linkage='ward'), moons, 222, 'Moons')  
fit_predict_plot(cluster.AgglomerativeClustering(n_clusters=3, linkage='ward'), blobs, 223, 'Blobs')  
fit_predict_plot(cluster.AgglomerativeClustering(n_clusters=3, linkage='ward'), no_structures, 224,
```

Hierarchical Clustering



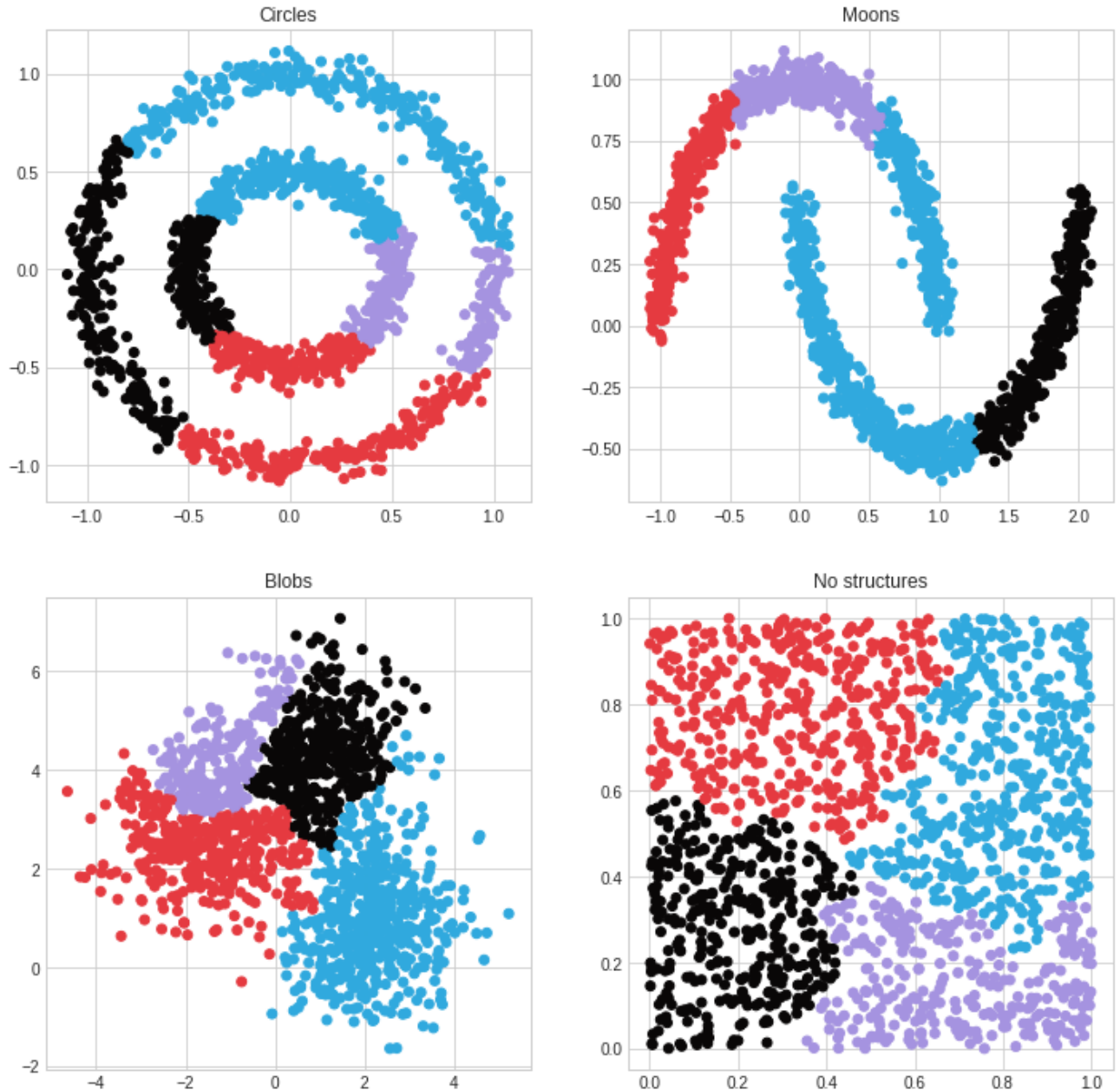
```
fig = plt.figure(figsize=(12, 12))  
fig.suptitle('Hierarchical Clustering')
```

```

fit_predict_plot(cluster.AgglomerativeClustering(n_clusters=4, linkage='ward'), circles, 221, 'Circles')
fit_predict_plot(cluster.AgglomerativeClustering(n_clusters=4, linkage='ward'), moons, 222, 'Moons')
fit_predict_plot(cluster.AgglomerativeClustering(n_clusters=4, linkage='ward'), blobs, 223, 'Blobs')
fit_predict_plot(cluster.AgglomerativeClustering(n_clusters=4, linkage='ward'), no_structures, 224,

```

Hierarchical Clustering



▼ 와인 데이터 군집화

```
from sklearn.datasets import load_wine
```

```
wine = load_wine()
```

```

model = cluster.AgglomerativeClustering(n_clusters=3)
model.fit(wine.data)
predict = model.labels_

```

```

idx = np.where(predict == 0)
wine.target[idx]

```

```

array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 1])

```

```

idx = np.where(predict == 1)
wine.target[idx]

```

```

array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2])

```

```

idx = np.where(predict == 2)
wine.target[idx]

```

```

array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])

```

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

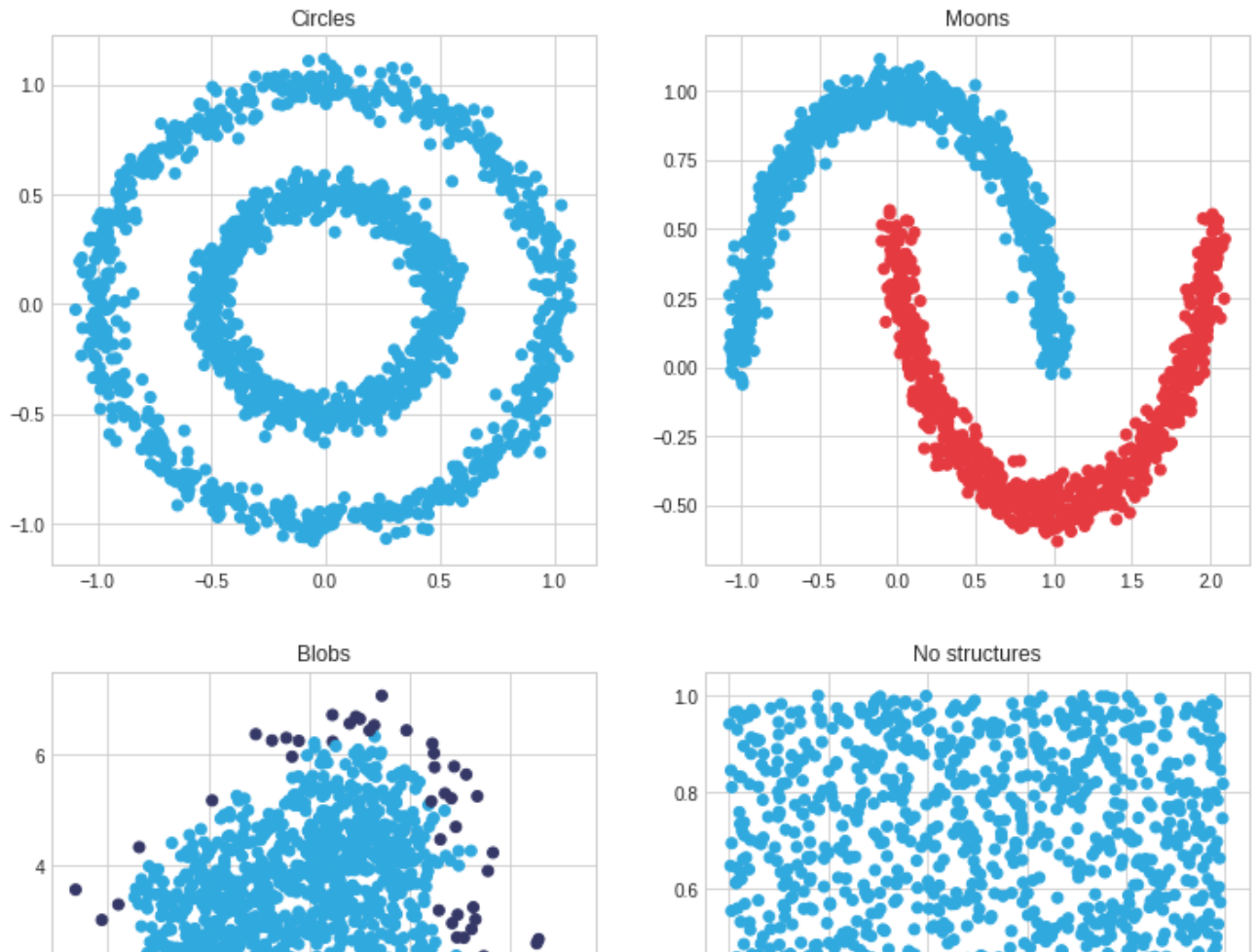
```

fig = plt.figure(figsize=(12, 12))
fig.suptitle('DBSCAN')

fit_predict_plot(cluster.DBSCAN(eps=.3), circles, 221, 'Circles')
fit_predict_plot(cluster.DBSCAN(eps=.3), moons, 222, 'Moons')
fit_predict_plot(cluster.DBSCAN(eps=.3), blobs, 223, 'Blobs')
fit_predict_plot(cluster.DBSCAN(eps=.3), no_structures, 224, 'No structures')

```

DBSCAN

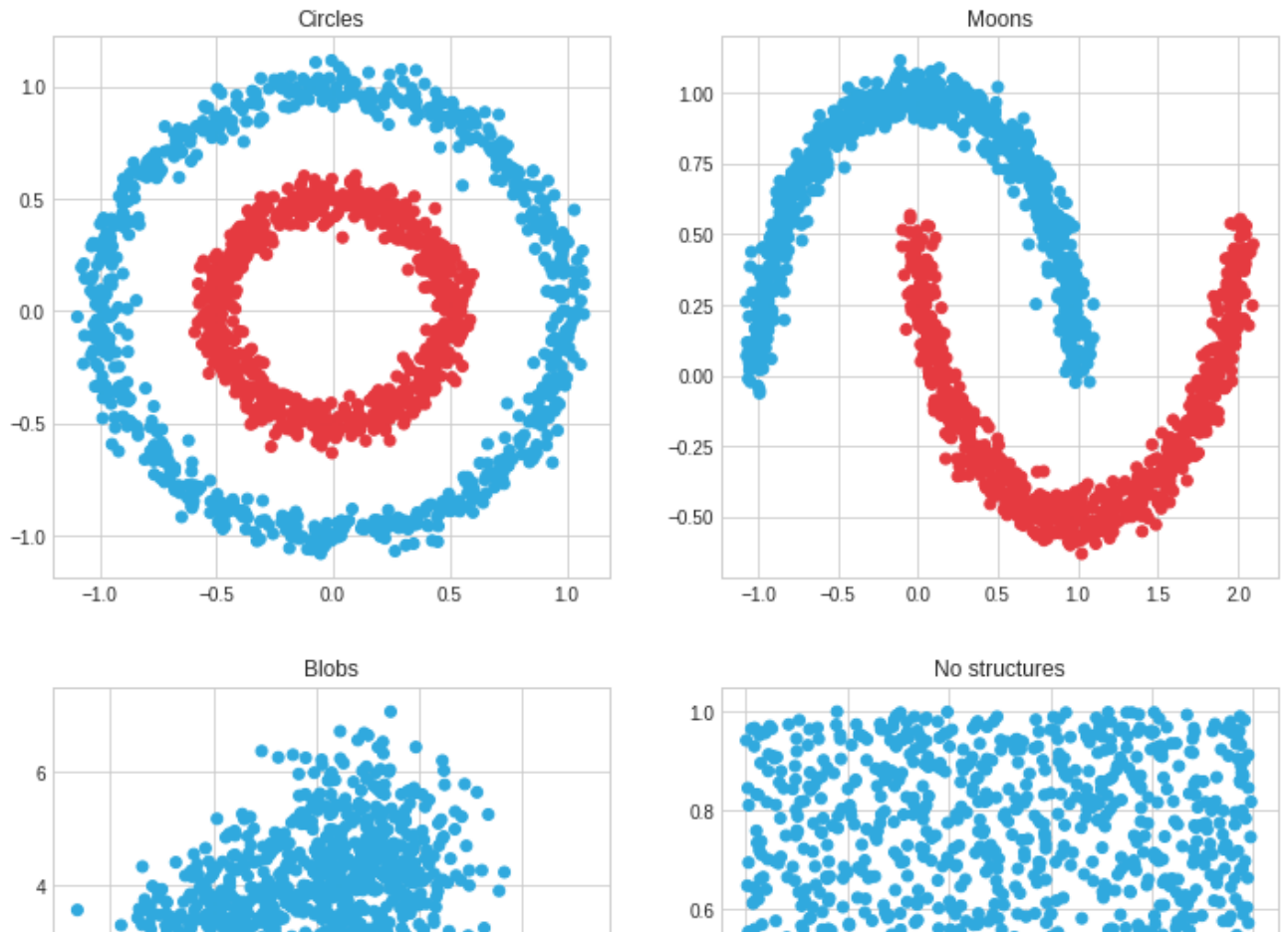


► OPTICS (Ordering Points To Identify the Clustering Structure)



```
fig = plt.figure(figsize=(12, 12))
fig.suptitle('OPTICS')
```

```
fit_predict_plot(cluster.OPTICS(min_samples=20, xi=0.05, min_cluster_size=0.1), circles, 221, 'Circles')
fit_predict_plot(cluster.OPTICS(min_samples=20, xi=0.05, min_cluster_size=0.1), moons, 222, 'Moons')
fit_predict_plot(cluster.OPTICS(min_samples=20, xi=0.05, min_cluster_size=0.1), blobs, 223, 'Blobs')
fit_predict_plot(cluster.OPTICS(min_samples=20, xi=0.05, min_cluster_size=0.1), no_structures, 224,
```

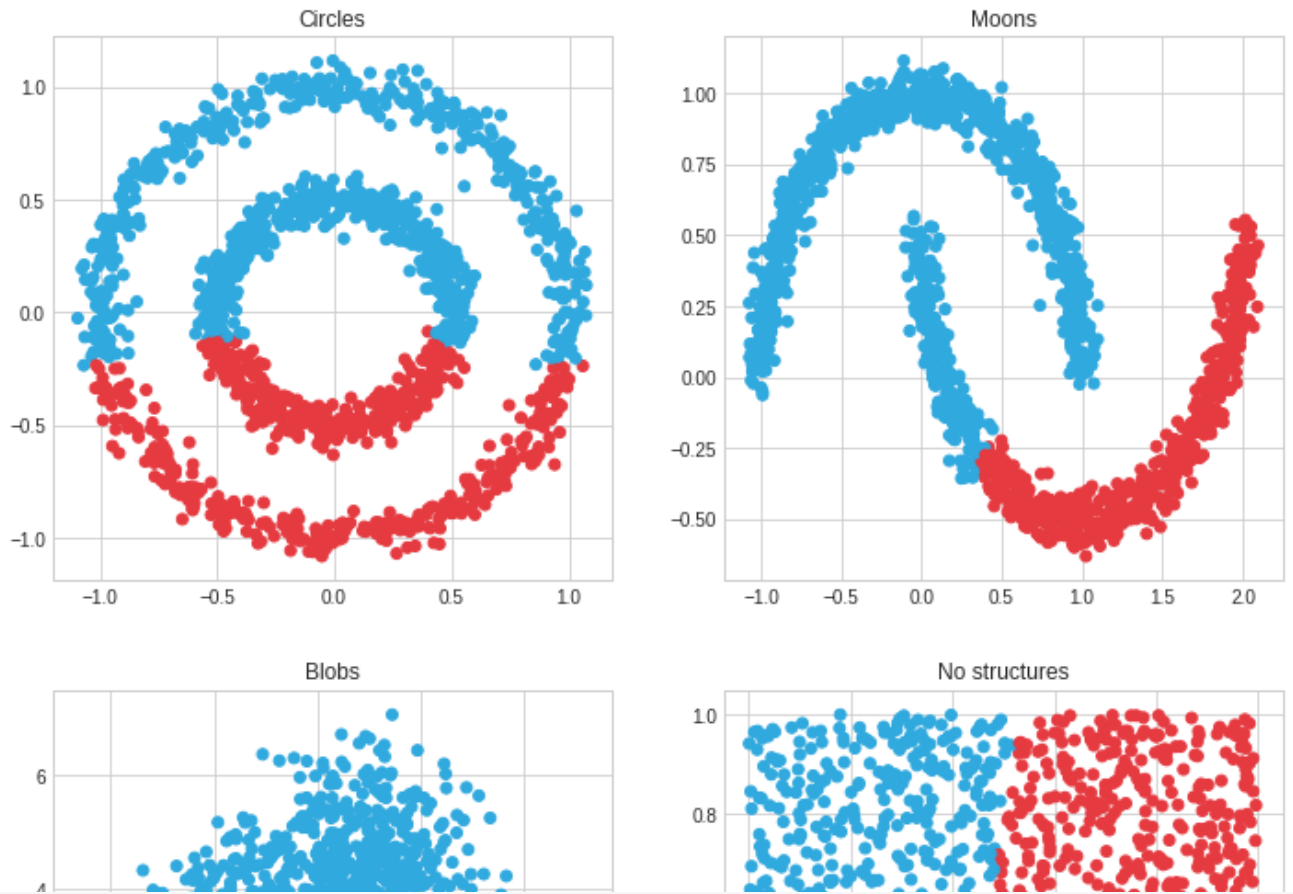


▼ Birch (Balanced iterative reducing and clustering using hierarchies)

```
fig = plt.figure(figsize=(12, 12))
fig.suptitle('Birch')

fit_predict_plot(cluster.Birch(n_clusters=2, threshold=.3), circles, 221, 'Circles')
fit_predict_plot(cluster.Birch(n_clusters=2, threshold=.3), moons, 222, 'Moons')
fit_predict_plot(cluster.Birch(n_clusters=2, threshold=.3), blobs, 223, 'Blobs')
fit_predict_plot(cluster.Birch(n_clusters=2, threshold=.3), no_structures, 224, 'No structures')
```

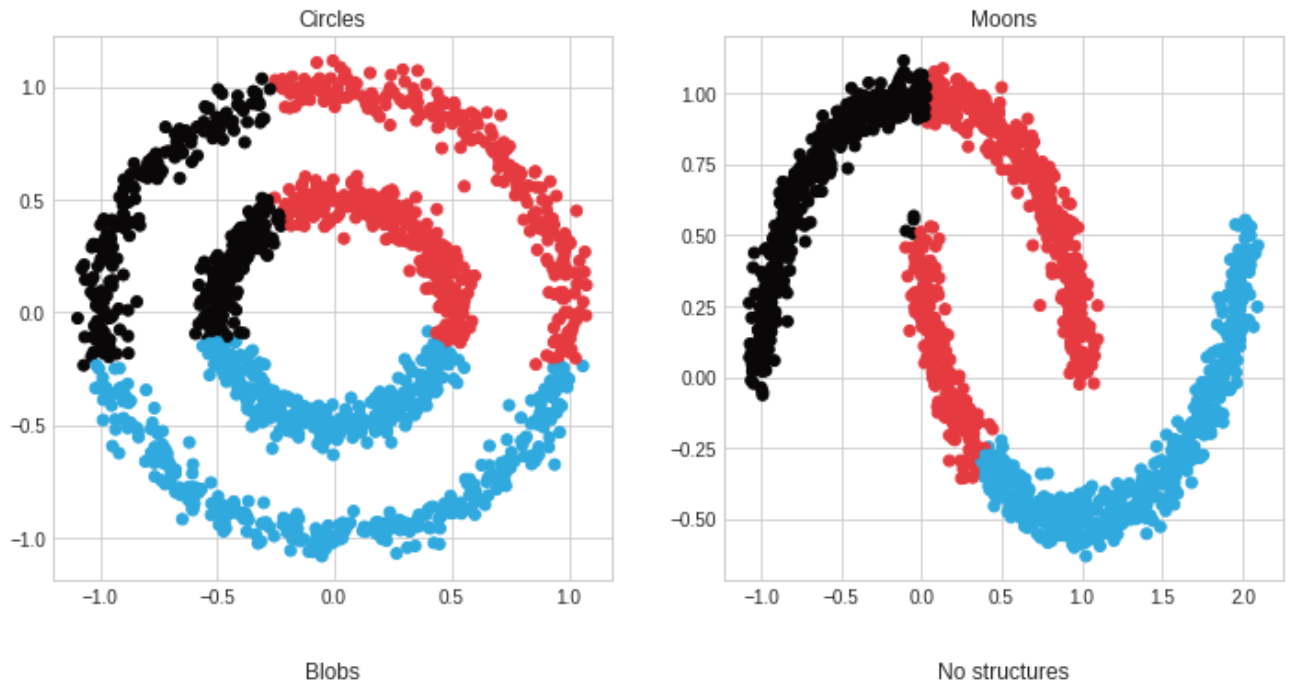
Birch



```
fig = plt.figure(figsize=(12, 12))  
fig.suptitle('Birch')
```

```
fit_predict_plot(cluster.Birch(n_clusters=3, threshold=.3), circles, 221, 'Circles')  
fit_predict_plot(cluster.Birch(n_clusters=3, threshold=.3), moons, 222, 'Moons')  
fit_predict_plot(cluster.Birch(n_clusters=3, threshold=.3), blobs, 223, 'Blobs')  
fit_predict_plot(cluster.Birch(n_clusters=3, threshold=.3), no_structures, 224, 'No structures')
```

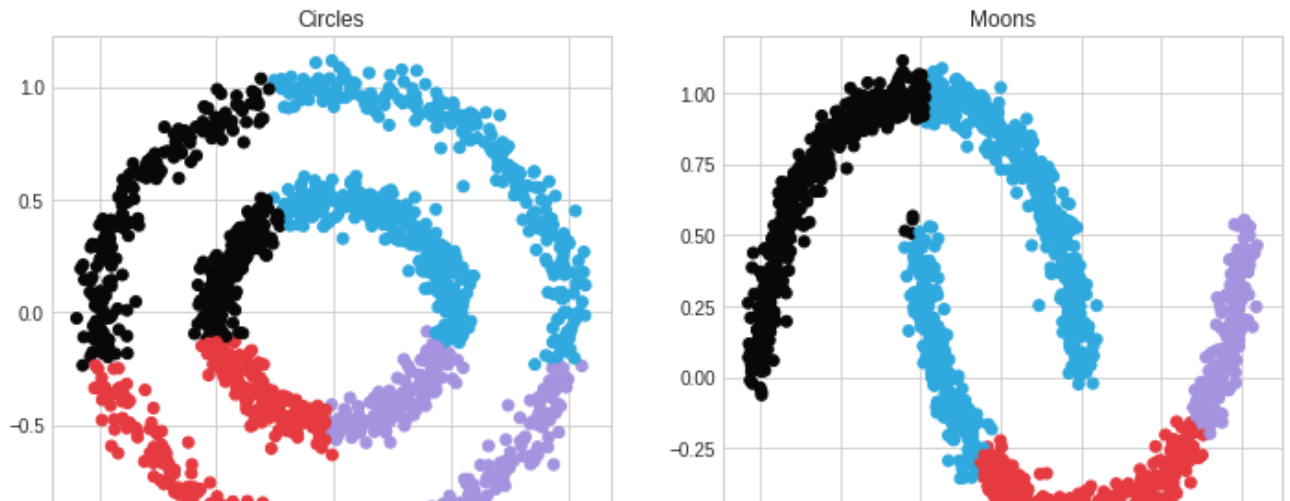
Birch



```
fig = plt.figure(figsize=(12, 12))
fig.suptitle('Birch')

fit_predict_plot(cluster.Birch(n_clusters=4, threshold=.3), circles, 221, 'Circles')
fit_predict_plot(cluster.Birch(n_clusters=4, threshold=.3), moons, 222, 'Moons')
fit_predict_plot(cluster.Birch(n_clusters=4, threshold=.3), blobs, 223, 'Blobs')
fit_predict_plot(cluster.Birch(n_clusters=4, threshold=.3), no_structures, 224, 'No structures')
```

Birch



▼ 손글씨 데이터 군집화

```
from sklearn.datasets import load_digits
```

```
digits = load_digits()
print(digits.DESCR)
```

```
.. _digits_dataset:
```

Optical recognition of handwritten digits dataset

****Data Set Characteristics:****

```
:Number of Instances: 5620
:Number of Attributes: 64
:Attribute Information: 8x8 image of integer pixels in the range 0..16.
:Missing Attribute Values: None
:Creator: E. Alpaydin (alpaydin '@' boun.edu.tr)
:Date: July; 1998
```

This is a copy of the test set of the UCI ML hand-written digits datasets
<https://archive.ics.uci.edu/ml/datasets/Optical+Recognition+of+Handwritten+Digits>

The data set contains images of hand-written digits: 10 classes where each class refers to a digit.

Preprocessing programs made available by NIST were used to extract normalized bitmaps of handwritten digits from a preprinted form. From a total of 43 people, 30 contributed to the training set and different 13 to the test set. 32x32 bitmaps are divided into nonoverlapping blocks of 4x4 and the number of on pixels are counted in each block. This generates an input matrix of 8x8 where each element is an integer in the range 0..16. This reduces dimensionality and gives invariance to small distortions.

For info on NIST preprocessing routines, see M. D. Garris, J. L. Blue, G. T. Candela, D. L. Dimmick, J. Geist, P. J. Grother, S. A. Janet, and C.

L. Wilson, NIST Form-Based Handprint Recognition System, NISTIR 5469, 1994.

.. topic:: References

- C. Kaynak (1995) Methods of Combining Multiple Classifiers and Their Applications to Handwritten Digit Recognition, MSc Thesis, Institute of Graduate Studies in Science and Engineering, Bogazici University.
- E. Alpaydin, C. Kaynak (1998) Cascading Classifiers, Kybernetika.
- Ken Tang and Ponnuthurai N. Suganthan and Xi Yao and A. Kai Qin. Linear dimensionality reduction using relevance weighted LDA. School of Electrical and Electronic Engineering Nanyang Technological University. 2005.
- Claudio Gentile. A New Approximate Maximal Margin Classification Algorithm. NIPS. 2000.

digits.data

```
array([[ 0.,  0.,  5., ...,  0.,  0.,  0.],
       [ 0.,  0.,  0., ..., 10.,  0.,  0.],
       [ 0.,  0.,  0., ..., 16.,  9.,  0.],
       ...,
       [ 0.,  0.,  1., ...,  6.,  0.,  0.],
       [ 0.,  0.,  2., ..., 12.,  0.,  0.],
       [ 0.,  0., 10., ..., 12.,  1.,  0.]])
```

digits.target

```
array([0, 1, 2, ..., 8, 9, 8])
```

```
X = digits.data.reshape(-1, 8, 8)
y = digits.target
```

```
plt.style.use(['seaborn-notebook'])
plt.figure(figsize=(16, 8))
for i in range(10):
    plt.subplot(2, 5, i+1)
    plt.imshow(X[i])
```




1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127	128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159	160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191	192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207	208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	223	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255	256	257	258	259	260	261	262	263	264	265	266	267	268	269	270	271	272	273	274	275	276	277	278	279	280	281	282	283	284	285	286	287	288	289	290	291	292	293	294	295	296	297	298	299	300	301	302	303	304	305	306	307	308	309	310	311	312	313	314	315	316	317	318	319	320	321	322	323	324	325	326	327	328	329	330	331	332	333	334	335	336	337	338	339	340	341	342	343	344	345	346	347	348	349	350	351	352	353	354	355	356	357	358	359	360	361	362	363	364	365	366	367	368	369	370	371	372	373	374	375	376	377	378	379	380	381	382	383	384	385	386	387	388	389	390	391	392	393	394	395	396	397	398	399	400	401	402	403	404	405	406	407	408	409	410	411	412	413	414	415	416	417	418	419	420	421	422	423	424	425	426	427	428	429	430	431	432	433	434	435	436	437	438	439	440	441	442	443	444	445	446	447	448	449	450	451	452	453	454	455	456	457	458	459	460	461	462	463	464	465	466
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

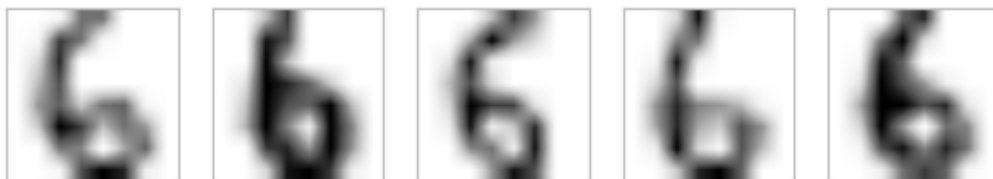
Cluster: 7

[illegible]

Cluster: 1



Cluster: 2



Cluster: 3



Cluster: 4



Cluster: 5



Cluster: 6



Cluster: 7



Cluster: 8



Cluster: 9



▼ Hierarchical Clustering

```
hierarchical = cluster.AgglomerativeClustering(n_clusters=10, linkage='ward')
hierarchical.fit(digits.data)
predict = hierarchical.labels_
```

```
for i in range(10):
    idx = np.where(predict==i)
    real_class = digits.target[idx]
    print('Cluster {}: {}'.format(i+1, real_class))
```

[illegible]

```

1 1 1 1 1 9 9 9 9 9 9 9 9 9 9 9 9 8 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 9 9 9 9 9]
Cluster 10: [1 1 1 8 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1]

```

```

for i in range(10):

    idx = np.where(predict == i)[0]
    choice_idx = np.random.choice(idx, size=5)
    choice_image = X[choice_idx]

    k = 1

    print('Cluster: {}'.format(i+1))
    for image in choice_image:
        plt.subplot(1, 5, k)
        plt.xticks([])
        plt.yticks([])
        plt.imshow(image)
        k += 1

    plt.show()

```


Cluster: 1



Cluster: 2



Cluster: 3



Cluster: 4



▼ Birch

```
birch = cluster.Birch(n_clusters=10, threshold=.3)
birch.fit(digits.data)
predict = birch.labels_
```

```
for i in range(10):
    idx = np.where(predict==i)
    real_class = digits.target[idx]
    print('Cluster {}: {}'.format(i+1, real_class))
```

[illegible]

Cluster: 1



Cluster: 2



Cluster: 3



Cluster: 4



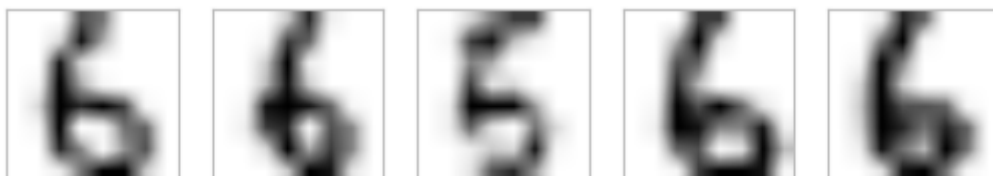
Cluster: 5



Cluster: 6



Cluster: 7



Cluster: 8



Cluster: 9



